

Ironline CPQ

System Documentation

Configure · Price · Quote on Kinetic

June 18, 2026

Server: ai-labs.kinopsdev.io · Kapp: cpq

Author: John Sundberg

164 quotes · 39 orders · 32 invoices

1 · Executive Summary

Ironline CPQ turns the configure-price-quote lifecycle of an industrial-equipment manufacturer into a fully instrumented quote-to-cash system on the Kinetic Platform. Sales reps configure multi-line quotes from a catalog of conveyor systems, pumps, control panels, robotic cells and safety systems; pricing rules apply volume and segment discounts; deals that breach a discount or margin threshold are routed for approval; won quotes convert into orders, invoices and payments.

The application is deliberately designed around one question: where does money move, and where does it get stuck? Every dashboard traces value from list price to collected cash and flags the four blockages that strangle quote-to-cash — stalled deals, approval bottlenecks, unconverted wins and aging receivables.

\$134.5M

Open Pipeline

\$67M

Won (booked)

75%

Win Rate

27%

Avg Margin

\$21.8M

Outstanding AR

Live blockage signal: 37 quotes stalled 30+ days, 13 approvals past SLA, 9 won deals (\$21M) never converted to orders, and \$21.8M in receivables still uncollected.

2 · Architecture & Data Model

The system is a single Kinetic kapp ("cpq") with ten linked forms. A thin Node service performs read-only aggregation for the dashboards; all business PROCESSES (pricing approval, conversion, invoicing) run as Kinetic workflow task trees on the platform itself — not in the web tier. This keeps the automation portable, auditable in the run log, and independent of the UI.

The ten forms

Form	Role in the money flow
products	Catalog: list price, unit cost, lead time, configurable flag
options	Configurable add-ons with price & cost deltas
price-rules	Volume tiers, segment discounts, margin floors, approval thresholds
customers	Accounts — segment, region, credit terms
quotes	Quote headers moving through the pipeline (the spine)

Form	Role in the money flow
quote-lines	Line items — qty, options, discount, per-line margin
approvals	Discount / margin-floor approval requests with SLA
orders	Won quotes converted to production orders
invoices	Billing against orders — the cash-conversion stage
payments	Receipts against invoices — closes the loop

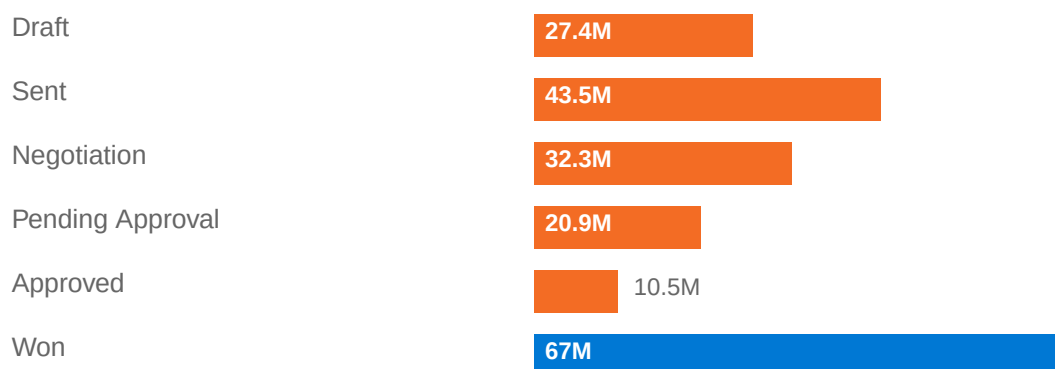
Design rule: processes are task trees, not web-server code. The Node service only reads and aggregates; it never mutates business state.

3 · The Four Money-Flow Visuals

The dashboard is built around four views, each pairing a flow with the blockage that interrupts it.

3.1 Pipeline Funnel — where deals stall

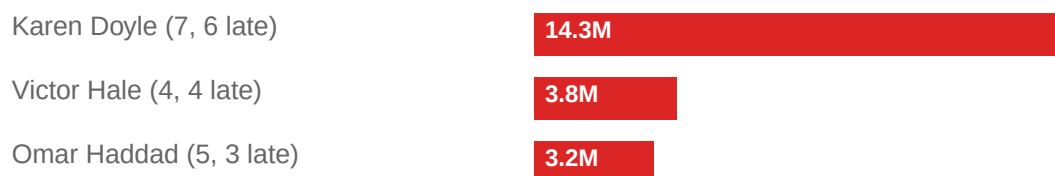
Open quotes by stage, bar width proportional to net value, with the portion that has not moved in 30+ days striped in red. It answers: how much value sits in each stage, and how much is rotting?



37 open quotes are currently stalled 30+ days in stage — the single largest source of slipped pipeline.

3.2 Approval Bottleneck — who is the blocker

Pricing approvals are the most common pipeline blockage. 16 approvals are pending (\$21.3M of value held), and 13 are already past their SLA. The board breaks the queue down by approver so the bottleneck has a name.



3.3 Revenue !' Margin Waterfall — where value leaks

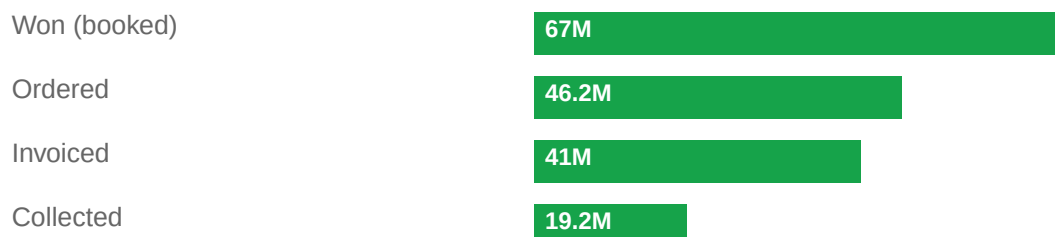
On won deals, list value is eroded first by discounts, then by cost of goods, leaving gross margin. The waterfall makes the leakage explicit.

Stage	Amount	Note
List value	\$74.6M	sum of catalog list prices on won deals
– Discounts	-\$7.7M	10% of list given away
= Net booked	\$67M	contracted revenue
– Cost of goods	-\$48.8M	unit cost + option cost
= Gross margin	\$18.2M	27% margin retained

Every point of discount on \$74.6M of list value is real money: discounts already cost \$7.7M on closed business this period.

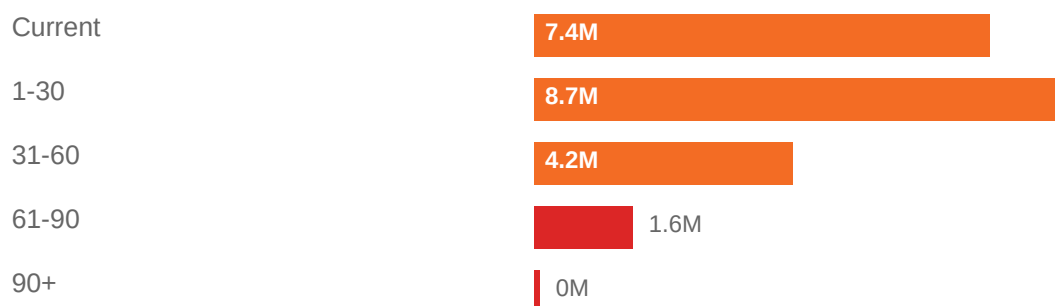
3.4 Cash Conversion — won money that is not yet cash

The final flow follows booked revenue all the way to the bank: Won !' Ordered !' Invoiced !' Collected, with the leak at each step called out.



Leak points: \$21M of won deals never became orders; \$5.2M is ordered but not yet invoiced; \$21.8M is invoiced but uncollected (6 invoices overdue).

Accounts-receivable aging



4 · Workflow Automation (Kinetic Task Trees)

Four event-triggered task trees automate the quote-to-cash process on the platform. Each calls the Core API through the `kinetic_core_api_connection_v1` handler; none of this logic lives in the web server. Together they form a self-driving pipeline: submit a quote and it routes, approves, converts and invoices itself.

4.1 Quote Approval Routing

Trigger: a quote is submitted (quotes / Submission Submitted). If discount > 15% or margin < 22%, the tree creates an approval request — choosing approver by deal size, type by which threshold broke, priority by discount depth, and an SLA due date three days out — then flags the quote "Pending Approval". Otherwise it marks the quote auto-approved.

Start

[illegible]

4.2 Approval Decision Sync

Trigger: an approval is updated (approvals / Submission Updated). The tree finds the parent quote by quote number and writes the decision back — "Approved" pushes the quote to Approved; "Rejected" returns it to Negotiation.

4.3 Won to Order Conversion

Trigger: a quote is updated to Status = Won with no order yet (quotes / Submission Updated). The tree creates a production order (SO-####) carrying the quote totals and cost, then links the order number back onto the quote — eliminating the "unconverted win" blockage automatically.

4.4 Order Invoice Generation

Trigger: an order is marked Shipped or Delivered (orders / Submission Updated). The tree raises an invoice for the order total with Net-30 terms and a due date, then records the invoiced amount on the order — starting the receivables clock.

All four trees were validated against the rule gate (node-id convention, lastID, installed handlers, server-root-relative paths) and verified end-to-end with live runs on ai-labs.

5 · Install & Usage

Provision the app

```
node apps/cpq/gen-seed.mjs                # regenerate deterministic seed
node apps/cpq/install.mjs https://ai-labs.kinopsdev.io john <pass> --seed
```

install.mjs creates the kapp, the ten forms, all search indexes (and waits for the builds), then loads the seed data. The app is auto-discovered by the base launcher on port 3011 and served at /cpq/.

Deploy the workflow trees

```
node apps/cpq/workflows/gen-trees.mjs      # emit treeXml + treeJson
node apps/cpq/workflows/put-trees.mjs      # validate + persist via Task API
```

Using the system

- Money Flow tab — the four visuals above, live.
- Pipeline / Approvals / Cash tabs — drill into each blockage.
- Quotes — click any quote for a 360 (lines, price build-up, approvals, fulfillment).
- Create a quote that breaks a threshold and watch the approval appear automatically.
- All tables are fully editable (row-click edit + Add New); edits fire the workflow trees.

Built and verified June 18, 2026 on ai-labs.kinopsdev.io. Dashboards render live data; all four automation trees confirmed firing on real submissions.